IFT3295 – TP1 : Assemblage de séquences

Réponses aux questions écrites

Toky Erick Rabenantoandro (20266409)

9 octobre 2026

1. Chevauchement de séquences

On note $v = X_i$ la séquence en lignes (taille m) et $w = X_j$ la séquence en colonnes (taille n). Le score d'une colonne d'alignement est $\delta(a, b) = +4$ si $a = b$, -4 si $a \neq b$, et -8 pour un indel. $V(i, j)$ est le meilleur score d'un alignement entre un suffixe de $v[1..i]$ et le préfixe $w[1..j]$.

Q1. Différence avec l'alignement global

Dans l'alignement global, on aligne les deux séquences au complet, donc les indels au début et à la fin sont pénalisés comme les autres. Pour un chevauchement, on veut seulement aligner la fin de v avec le début de w (alignement suffixe-préfixe). Le début de v et la fin de w ne font pas partie du chevauchement : ils ne doivent donc pas être pénalisés.

Q2. Première ligne et première colonne

$$V(i, 0) = 0 \quad \forall i \qquad\qquad V(0, j) = -8\, j \quad \forall j$$

- Première colonne : le chevauchement peut commencer n'importe où dans v. Sauter le préfixe $v[1..i]$ ne coûte rien, donc on met 0 (comme un « parcours gratuit » en alignement local).
- Première ligne : le chevauchement doit commencer au premier caractère de w. Si on a déjà utilisé j caractères de w sans aucun caractère de v, ce sont forcément j indels, donc $-8j$.

Q3. Relations de récurrence

Pour $1 \leq i \leq m$ et $1 \leq j \leq n$, on prend le meilleur des trois cas habituels :

$$V(i, j) = \max \begin{cases} V(i-1, j-1) + \delta(v_i, w_j) & v_i \text{ aligné avec } w_j \\ V(i-1, j) - 8 & v_i \text{ aligné avec un indel} \\ V(i, j-1) - 8 & w_j \text{ aligné avec un indel} \end{cases}$$

C'est la même récurrence que pour l'alignement global : seules l'initialisation et la case finale changent. On garde aussi, pour chaque case, un pointeur vers le cas choisi. La table se remplit ligne par ligne en $O(mn)$.

Q4. Retrouver l'alignement

1. Le chevauchement doit finir à la fin de v, donc on regarde la dernière ligne. On choisit $j^* = \arg\max_j V(m, j)$; le score du chevauchement est $V(m, j^*)$. S'il est ≤ 0, il n'y a pas de chevauchement.
2. On part de la case (m, j^*) et on suit les pointeurs : diagonale ⇒ on aligne v_i avec w_j ; haut ⇒ v_i avec « - » ; gauche ⇒ « - » avec w_j.
3. On s'arrête quand $j = 0$: tout le préfixe de w a été aligné et le reste de v est ignoré.
4. Comme on a construit l'alignement à l'envers, on l'inverse. La longueur du chevauchement est le nombre de colonnes de l'alignement.

Q5. Implémentation

L'algorithme est dans `overlap.py` (`chevauchement_maximal`), en $O(|X_i||X_j|)$. Pour les deux premiers reads de `reads.fq`, on obtient un score de 8, l'alignement `TTAG/CTAG` et un chevauchement de longueur 4.

2. Assemblage de fragments

2.1 Matrice des scores

La case (ligne R_i, colonne R_j) donne le score du chevauchement entre la fin de R_i et le début de R_j. La matrice n'est pas symétrique et sa diagonale est nulle. Les scores ≥ 80 sont en gras.

$R_i \backslash R_j$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	0	8	8	0	4	12	12	16	4	**804**	20	4	0	4	**396**	4	20	**652**	12	**608**
2	4	0	8	12	44	8	4	**416**	8	12	**424**	16	0	**1060**	12	12	12	8	4	0
3	**396**	12	0	24	**1016**	16	48	0	36	**128**	4	20	40	8	20	20	16	8	8	8
4	28	8	28	0	12	**640**	**1024**	16	**1068**	12	12	**220**	16	24	12	**656**	**152**	16	8	16
5	**500**	4	**696**	28	0	24	52	8	52	**208**	16	20	36	4	8	24	36	64	4	12
6	52	4	4	16	0	0	4	4	12	28	4	16	4	0	28	8	**672**	16	12	20
7	32	12	32	**648**	16	**720**	0	4	**772**	12	12	24	24	24	24	**176**	**232**	12	4	0
8	8	12	**644**	4	**540**	16	8	0	8	4	**944**	12	4	12	0	4	8	36	4	16
9	36	12	20	**956**	16	**668**	**1044**	4	0	8	4	64	12	24	20	**492**	**196**	8	4	12
10	4	0	0	8	32	8	8	24	12	0	32	4	12	12	**640**	4	8	**904**	**184**	**868**
11	20	20	**620**	4	**524**	12	12	**1072**	4	12	0	24	28	24	8	8	16	20	12	12
12	28	28	48	**900**	16	**460**	**820**	0	**856**	4	16	0	28	24	16	**996**	20	16	8	4
13	68	32	32	**744**	24	**272**	**656**	0	**692**	16	8	**860**	0	36	16	**848**	32	24	4	28
14	16	**756**	12	12	52	20	8	**452**	12	16	**492**	16	8	0	4	12	20	16	8	16
15	20	8	28	12	16	12	16	12	8	4	12	**80**	**260**	28	0	28	20	24	**632**	24
16	20	16	24	**1032**	16	**552**	**944**	12	**988**	8	20	**692**	8	20	12	0	**80**	12	8	8
17	16	4	32	16	16	32	8	12	4	24	20	12	4	28	12	8	0	16	8	20
18	16	0	8	12	8	4	4	8	8	**240**	16	8	4	16	**808**	8	4	0	**368**	**1004**
19	48	20	16	**388**	8	20	**316**	8	**368**	16	20	**520**	**732**	16	24	**492**	20	20	0	4
20	8	4	0	4	0	8	4	0	0	0	4	0	64	8	**908**	4	16	**644**	**428**	0

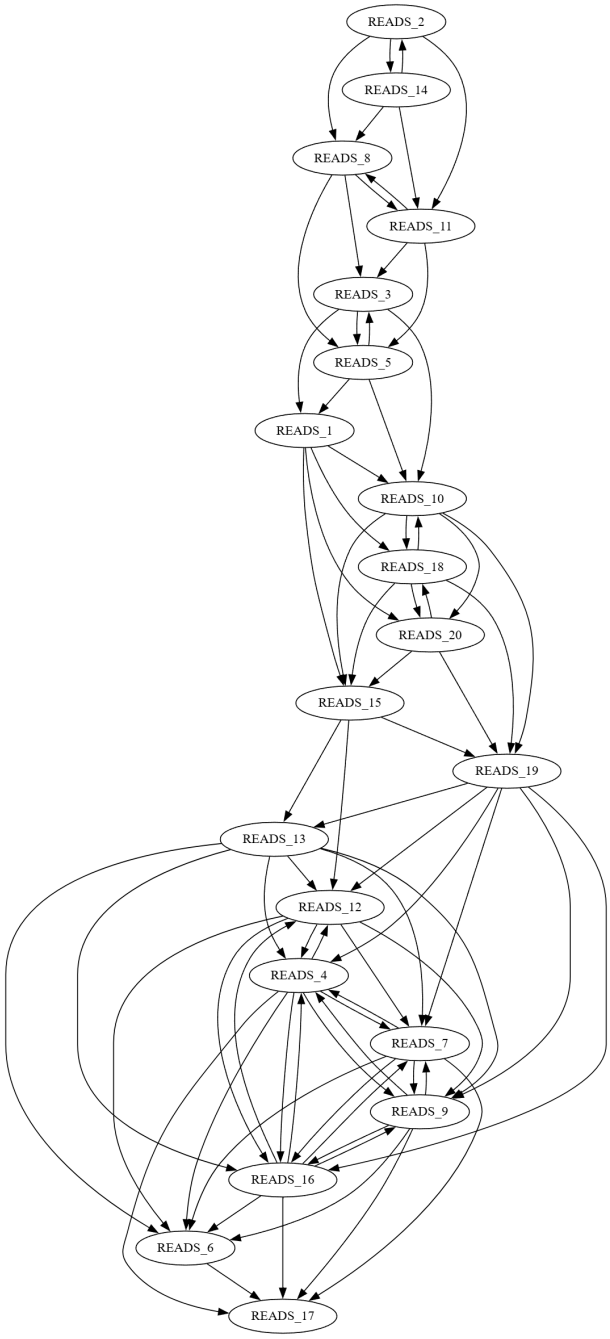
2.2a Effet du seuil de 80

Sans seuil, presque toutes les paires de reads ont un petit score positif, parce que quelques bases peuvent se ressembler par hasard : le graphe est presque complet et inutilisable. Le seuil de 80 enlève ces faux chevauchements et ne garde que les arêtes entre reads qui se recouvrent vraiment. Le graphe devient beaucoup plus simple, mais il contient encore des arêtes redondantes (transitives) et des cycles de deux noeuds, qu'il faut enlever à l'étape suivante.

En chiffres (calculés sur `overlap_matrix.csv`) : sans seuil, 360 paires ont un score positif ; avec le seuil, il reste 76 arêtes. Le plus grand score éliminé est 68, alors que les chevauchements gardés dans l'ordre

final valent tous au moins 492. Il reste 13 cycles de deux noeuds.

La figure ci-dessous montre le graphe filtré (seuil ≥ 80). Elle a été générée à partir du fichier `graphe.dot` produit par `main.py`, avec le site GraphvizOnline (`https://dreampuf.github.io/GraphvizOnline`, moteur `dot`).



2.2b Réduction transitive

La réduction se fait en deux étapes :

1. **Cycles de deux noeuds.** Quand deux reads ont une arête dans chaque sens, on garde seulement celle de plus grand score. Par exemple, pour READS_2 $\leftrightarrow$ READS_14, on garde READS_2 $\rightarrow$ READS_14 (1060) et on enlève READS_14 $\rightarrow$ READS_2 (756). Il y a 13 cycles de ce type.

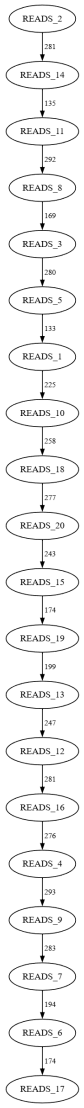
2. **Arêtes transitives.** On enlève une arête (u, v) s'il existe encore un autre chemin de u à v. Par exemple, READS_2 → READS_11 est enlevée, car le chemin READS_2 → READS_14 → READS_11 existe déjà. On enlève ainsi 44 arêtes.

Il reste 19 arêtes qui forment un seul chemin passant par les 20 reads, ce qui donne l'ordre :

READS_2, 14, 11, 8, 3, 5, 1, 10, 18, 20, 15, 19, 13, 12, 16, 4, 9, 7, 6, 17

Le graphe réduit ci-dessous a été généré avec GraphvizOnline (`https://dreampuf.github.io/GraphvizOnline`) ; l'étiquette de chaque arête est la longueur du chevauchement entre les deux reads.

Paire consécutive	Score	Chevauchement (nt)
READS_2 → READS_14	1060	281
READS_14 → READS_11	492	135
READS_11 → READS_8	1072	292
READS_8 → READS_3	644	169
READS_3 → READS_5	1016	280
READS_5 → READS_1	500	133
READS_1 → READS_10	804	225
READS_10 → READS_18	904	258
READS_18 → READS_20	1004	277
READS_20 → READS_15	908	243
READS_15 → READS_19	632	174
READS_19 → READS_13	732	199
READS_13 → READS_12	860	247
READS_12 → READS_16	996	281
READS_16 → READS_4	1032	276
READS_4 → READS_9	1068	293
READS_9 → READS_7	1044	283
READS_7 → READS_6	720	194
READS_6 → READS_17	672	174



2.2c Séquence du fragment

On écrit READS_2 au complet, puis pour chaque read suivant on ajoute seulement la partie qui dépasse le chevauchement. En cas de mismatch dans le chevauchement, on garde la base du read précédent. La longueur est $20 \times 300 - 4414 = \mathbf{1586}$ nucléotides (aussi dans `fragment.fasta`).

```
CTAGGGACTTGGAGAACACAAGTATTATGAAAAGTACTGATGAAAGTTATTAACAGGTTTCGAAAAATAACTTTACTATC
TGACGTGTTGCTTCTGCCGAGGATGACCGTTATTCCTGGTTTTGCATTTATATTTCACGTATGGTTAAATGTGCCAGCGT
```

```
TGTGGTTTAAAACTAATAGTAATAATATGCTTCTTTGTTCAGTTGGCTAGAGATTTACTACATCCGTCCTTGGAAGAGGA
AAAGAAAAAACATAACAAGAAACGCCTAGTACAAAGTCCAAATTCTTACTTTATGGATGTAAAATGTCCAGGTAAAATTT
GAAATCTTAATTCCTTTACTAAAGAAAATTTCTGTAGGGATTGCTAGTGTGGTGTGTATAGTTAAGATACATTAGAATCC
TCTGTTGAGTAGAAGTGGGATTACAGAATTGGACATGTCAGGGACAATTTCATAATAAATGTACTGAAACCATTGTAGAA
ACATTTCGGGGTAAGTGAAAACAGGCATTCAAAGTAGATATGCCATTTGAGTGCACTTGTGGCTAATCAAACTGTGGATA
CTCAAAATAGAGATGTTCTTTAATTGTAAAATCCTCACTGGATTTTGACGATGTAGCACAGAAAAAAAATACATTGATTA
CACTGTTTTTAAAAAATTTTGTGTCGCTGCTAGAAAAGTTTAAGTTACACATGTGGCTTGGTGTTTCATAGCACTGAAGT
TACTGATTTTTTTACATATTACCAGAAATGTCGAACAGGTGCTGTTTTCCTCTGCTTTTCATTTTTAAGATTGCCTTTTT
TTTTTTTTAGGTTGCTACAAGATCACCACGGTTTTCAGCCATGCTCAGACTGTGGTTCTTTGTGTAGGTTGTGCAACAGT
GTTGTGCCAGCCTACAGGAGGAAAGGCCAGACTCACAGAAGGTATATCATTTGGCATTCTCCAACCCAGTGATGAGATTG
ATGATTTTAAATGTCTCTATATTAACTGAAAAGTTTAAAGAAATCTTAATGATTCCCAAAATAACTTATCTCACACTGGA
AGAGTTCAAGTGGATTGGCAGCAAATCTGAGATCTATTTGGTGTGACCTGGTGAGATCTAAATATGGAGTCAGCACATGA
TTTTTTTAAGAGTAATATTGCTAAGTAATATTGCTAAGTATCGTCTGAAAATACCTCTAATCAAAATTATTTACTTGAGA
AAAGTATTCAGAATAGTTCCTAAAAATTAATAGTATATTTCTGGTATATAAGCATAAATAATCTGTATATGAGTATTAAT
CCAATATTCTTAAAACTTCAGTATTTTACTTAAAAGTACTATTTGTCATTAAAATTATACCAAAGGTAGAATGCACTTGT
TTAATATACTCTCATGATTCTTTTGCAGGGTGTTCATTTAGAAGAAAGCAACACTAATGATTCAAACAGCTTCCTGAATT
TTAATTTTGTGTTGTCTCTCAGAAAGCCTTATCATAAATTCCATAATTCTAATTAATTTACCAAGATAATGTCATTACAT
TTGGTTATGTAAGTTATACAGCAGTAATCTCCTATTTTGGTGTCAGTTTTTCACTAAAGTTTTTAT
```

3. Recherche d'introns et Blast

1a. Cadre de lecture

On a traduit `sequence.fasta` dans les trois cadres de lecture avec le code génétique standard. Le début de la protéine (`MCQRCGLKL...`) apparaît dans le **cadre 2** : le codon start ATG est aux positions 200 à 202.

1b. Algorithme de programmation dynamique

On aligne la protéine $P[1..m]$ avec la séquence génomique $S[1..n]$ lue codon par codon : $T(j)$ est l'acide aminé codé par $S[j..j+2]$. Dans un exon, chaque acide aminé correspond à exactement un codon ; on permet les mismatchs (coût $\mu = 1$) mais pas les indels. Un intron est un saut entre deux codons non consécutifs. Comme on veut minimiser les introns, on donne un coût constant ρ (ici 3) à chaque intron, peu importe sa longueur (pondération constante des gaps vue en cours).

$A(i, j)$ = coût minimal pour aligner $P[1..i]$ avec l'acide aminé i sur le codon qui commence en j.

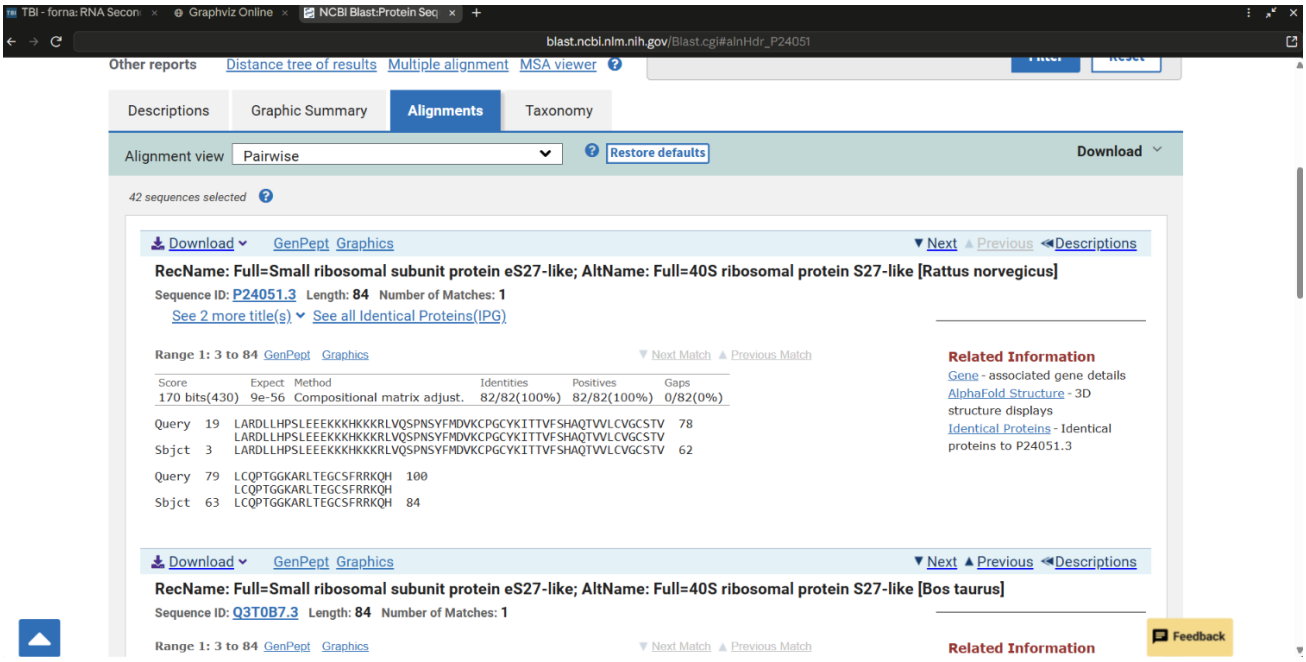
- **Conditions initiales :** $A(1, j) = \mu(P[1], T(j))$ pour tout j, car la région avant le gène ne coûte rien.
- **Récurrence :**

$$A(i,j) = \mu(P[i], T(j)) + \min\Big(\underbrace{A(i-1, j-3)}_{\text{même exon}},\ \underbrace{\rho + \min_{k \le j-4} A(i-1,k)}_{\text{nouvel intron}}\Big)$$

  Le premier cas continue l'exon avec le codon juste avant ; le second ouvre un intron entre le codon k et le codon j. En gardant le minimum courant de la ligne $i-1$, chaque case se calcule en $O(1)$, donc l'algorithme est en $O(mn)$. Si on impose que tous les exons soient dans le même cadre, on prend seulement les $k \equiv j \pmod 3$.
- **Chemin :** la meilleure valeur est $\min_j A(m, j)$. On remonte les pointeurs : un pas $j \to j-3$ reste dans le même exon, un saut $j \to k$ correspond à un intron.

2. Blastp

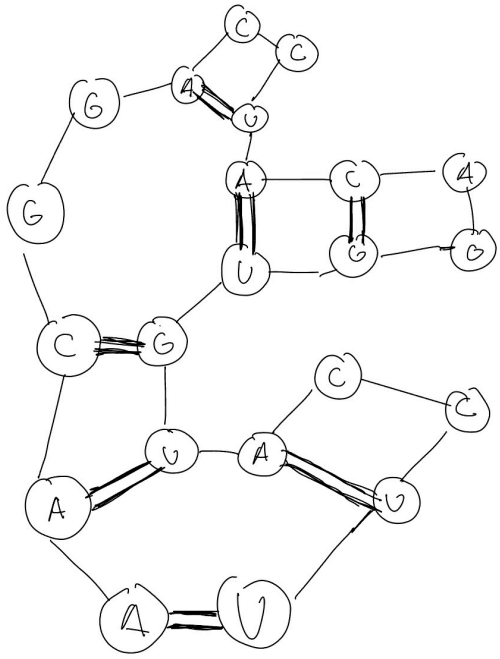
On a soumis la séquence de `geneX.fasta` à Blastp (NCBI). Le meilleur alignement est avec la protéine **40S ribosomal protein S27-like** (*small ribosomal subunit protein eS27-like*, gène **RPS27L**) : 82 acides aminés identiques sur 82 (100 %), sans gap, avec une E-value de 9×10^{-56}. L'alignement couvre les positions 19 à 100 de la protéine X ; seuls les 18 premiers acides aminés ne sont pas alignés. Les premiers résultats sont la même protéine chez plusieurs mammifères (rat, bœuf, …), ce qui montre qu'elle est très conservée.



Fonction : RPS27L est une protéine ribosomique de la famille de RPS27 (petite sous-unité 40S du ribosome). Son gène est une cible directe de p53 : elle est produite après des dommages à l'ADN, favorise l'apoptose et participe à la régulation de l'axe p53–MDM2.

4. Repliement d'ARN

Q7. Repliement 2D



Q8 (bonus). Maximiser le nombre d'empilements en $O(n^3)$

Un empilement est une paire (i, j) telle que $(i+1, j-1)$ est aussi appariée. On adapte l'algorithme de Nussinov avec deux tables ($p(x, y) = 1$ si x–y est A–U ou G–C) :

- $D(i, j)$: nombre maximal d'empilements pour $S[i..j]$;
- $E(i, j)$: même chose, mais en imposant que i et j soient appariés ($-\infty$ si $p(S_i, S_j) = 0$).

Conditions initiales : $D(i, i) = D(i, i-1) = 0$ et $E(i, j) = 0$ si $j - i \leq 1$.

Récurrences :

$$E(i,j) = \max \begin{cases} D(i+1, j-1) & (i,j) \text{ n'est pas empilée} \\ E(i+1, j-1) + 1 & \text{si } p(S_{i+1}, S_{j-1}) = 1 \text{ (empilement)} \end{cases}$$

$$D(i,j) = \max \begin{cases} D(i+1, j) & i \text{ non apparié} \\ D(i, j-1) & j \text{ non apparié} \\ E(i,j) & i \text{ apparié avec } j \\ \max_{i+1 \leq k \leq j-2} [D(i,k) + D(k+1, j)] & \text{bifurcation} \end{cases}$$

On remplit diagonale par diagonale et la réponse est $D(1, n)$. Pour retrouver le repliement, on part de $D(1, n)$ et on regarde quel cas a donné la valeur : dans E on note la paire (i, j), et une bifurcation donne deux sous-intervalles à traiter (avec une pile). Il y a $O(n^2)$ cases et la bifurcation coûte $O(n)$ par case, donc l'algorithme est en $O(n^3)$ en temps et $O(n^2)$ en espace.